QATIP Intermediate AWS Lab 05
Validations and Role-Based Access in AWS

Contents

Overview

This lab introduces Terraform validation mechanisms (preconditions and postconditions) and enforces secure, least-privilege IAM role usage through AWS AssumeRole mechanisms. You'll switch between users, roles, and credentials to explore how access is controlled and verified during resource deployment.

Before you begin

1. Ensure you have completed Lab0 before attempting this lab.
2. In the IDE terminal pane, enter the following command...

   cd ~/environment/aws-tf-int/labs/05

3. This shifts your current working directory to labs/05. Ensure all commands are executed in this directory

4. Close any open files and use the Explorer pane to navigate to and open the labs/05 folder.

Phase 0 – Setup: Create a Limited Test User and IAM Role

Purpose

- Create a new IAM user, John, and define a role it can assume, with limited permissions.

Steps Summary

- Create test user “John” and assign a password (for console access if needed).
- Create a role TerraformLimitedAccessRole with permissions limited to EC2 and S3 using an existing policy.
- Define a trust policy to allow John to assume this role.
- Attach an assume-role policy to John.

Rationale

- By creating a constrained test environment, this phase simulates real-world identity-based access where users can't directly perform privileged actions but must assume authorized roles.

Steps

1. Confirm your current identity:

   aws sts get-caller-identity

```
student:~/environment $ aws sts get-caller-identity
{
    "UserId": "AIDA57W2SBO7NYBSVPUXR",
    "Account": "961457294270",
    "Arn": "arn:aws:iam::961457294270:user/student"
}
```

2. Create a new IAM user called 'John':

   aws iam create-user --user-name John

```
{tudent:~/environment $
    "User": {
        "Path": "/",
        "UserName": "John",
        "UserId": "AIDA57W2SBO7LCS7AKSB6",
        "Arn": "arn:aws:iam::961457294270:user/John",
        "CreateDate": "2026-06-14T07:45:48+00:00"
    }
}
```

3. Grant console access to John — not strictly required for Terraform

```
aws iam create-login-profile \
 --user-name John \
 --password 'TestPassword123!' \
 --no-cli-pager
```

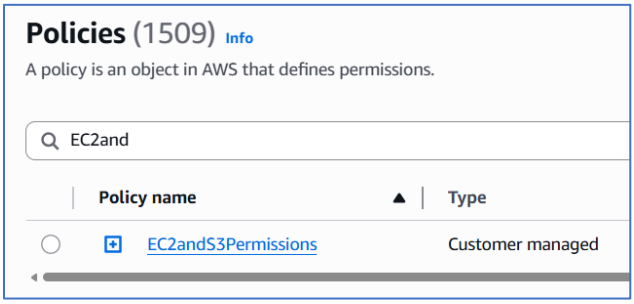
```
{
    "LoginProfile": {
        "UserName": "John",
        "CreateDate": "2026-06-14T07:49:42+00:00",
        "PasswordResetRequired": false
    }
}
```

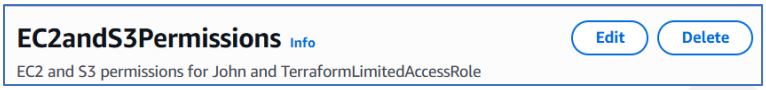
4. Create a new IAM role '**TerraformLimitedAccessRole**' and grant **John** the right to assume the role using the provided **trust-policy.json** file:

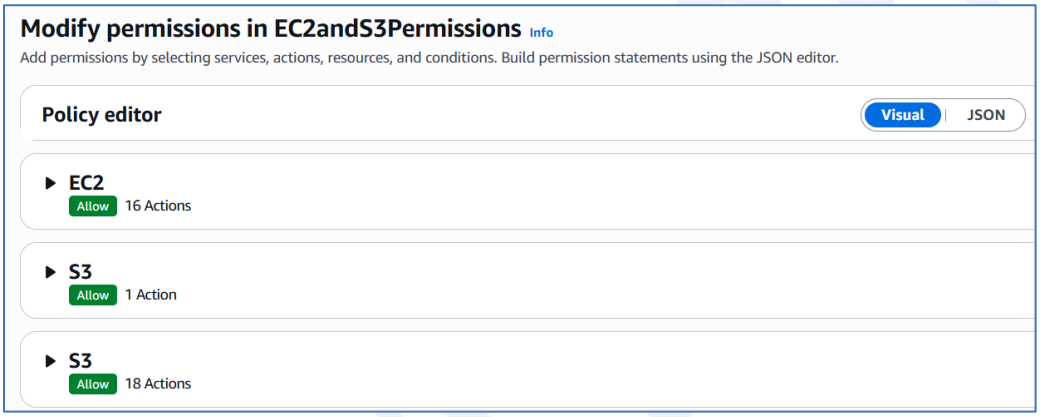
```
aws iam create-role \
 --role-name TerraformLimitedAccessRole \
 --assume-role-policy-document file://trust-policy.json \
 --no-cli-pager
```

```
student:~/environment/aws-tf-int/labs/05 (main) $ aws iam create-role \
>   --role-name TerraformLimitedAccessRole \
>   --assume-role-policy-document file://trust-policy.json \
>   --no-cli-pager
{
    "Role": {
        "Path": "/",
        "RoleName": "TerraformLimitedAccessRole",
        "RoleId": "AROA57W2SBO7G5EFJUW5N",
        "Arn": "arn:aws:iam::961457294270:role/TerraformLimitedAccessRole",
        "CreateDate": "2026-06-14T08:04:18+00:00",
        "AssumeRolePolicyDocument": {
            "Version": "2012-10-17",
            "Statement": [
                {
                    "Effect": "Allow",
                    "Principal": {
                        "AWS": "*"
                    },
                    "Action": "sts:AssumeRole",
                    "Condition": {
                        "ArnLike": {
                            "aws:PrincipalArn": [
                                "arn:aws:iam::*:user/student",
                                "arn:aws:iam::*:user/John"
                            ]
                        }
                    }
                }
            ]
        }
    }
}
```

5. In the AWS console, review the provided policy **EC2andS3Permissions**:







6. Attach this managed policy to the role you have just created:

```
aws iam attach-role-policy \
 --role-name TerraformLimitedAccessRole \
 --policy-arn arn:aws:iam::$(aws sts get-caller-identity --query
Account --output text):policy/EC2andS3Permissions \
 --no-cli-pager
```

Phase 1 – Adopt the test user Identity

Purpose

- Temporarily adopt the identity of the test user ‘John’ to experience the access restrictions firsthand.

Steps Summary

- Generate and configure access keys for testuser.
- Confirm identity with aws sts get-caller-identity.

Rationale

- This phase ensures Terraform will operate under the limited user context, which becomes important for demonstrating permission denials and later, proper role assumption.

Steps

1. Generate access keys for 'John':

   ```
   aws iam create-access-key \
    --user-name John \
    --no-cli-pager
   ```

2. Note down the AccessKeyId and SecretAccessKey
3. Configure AWS CLI to use testuser credentials:

   aws configure

   AWS Access Key ID: **(Access key)**
   AWS Secret Access Key: **(Secret key)**
   Region: **us-east-1**
   Output format: (leave blank)

4. Verify these are your current credentials:

   aws sts get-caller-identity

5. Terraform will now run under this identity

Phase 2 - Assume the Role

Purpose

- Observe what happens when a user attempts to deploy resources without the appropriate permissions.

Steps Summary

- Populate terraform.tfvars with randomized identifiers.
- Run terraform init and terraform apply as test user “John”
- Expect permission errors for EC2 and S3 operations.

- Investigate the main.tf file and uncomment the role block to enable AssumeRole.
- Expect success for EC2 and S3 operations.

Rationale

This phase highlights how assuming roles is required for privilege escalation and how policy updates must be done from a privileged account.

Steps

1. Update terraform.tfvars, replacing <random-6-digits> with 6 random digits.
2. Run **terraform init**
3. Run **terraform plan** – planning should succeed
4. Run **terraform apply** - 2 permissions errors should be generated (ec2:RunInstances and s3:CreateBucket)
5. Uncomment lines 5 through 8 of **main.tf** and update line **6** with your account ID
6. Run **terraform apply** – The deployment now progresses successfully
7. Tidy up with **terraform destroy**

Phase 3 – EC2 Instance Type Precondition Check

Purpose

- Demonstrate how Terraform can enforce guardrails to prevent undesired configurations.

Steps Summary

- Modify instance type to m5.large in terraform.tfvars.
- Run terraform plan — observe failure due to precondition.
- Revert to t2.small — observe planning success.

Rationale

- Preconditions help enforce organizational standards before provisioning happens. In this case, instance types not beginning with t2 are disallowed.

Steps

1. Update **terraform.tfvars** to attempt to provision an EC2 instance of type 'm5.large'.
2. Run **terraform plan**
3. Terraform planning fails with a validation error (precondition). The precondition checks that the variable **var.instance_type** begins with "t2"
4. Update terraform.tfvars, reverting the instance type to **“t2.small”**
5. Run **terraform plan**
6. The planning is successful as the variable **var.instance_type** now meets the precondition.

Phase 4 – Bucket Name Validations (Precondition and Postcondition)

Purpose

- Explore the difference between validating input variables (precondition) and verifying actual outcomes (postcondition) after logic is applied.

Steps Summary

- Attempt to create an S3 bucket that violates a precondition.
- Modify the bucket parameters to meet the precondition.
- Introduce a logic error to invoke a postcondition failure.
- Undo the error to meet the postcondition.
- Apply the deployment and verify resources are created.
- Clean up by destroying resources.

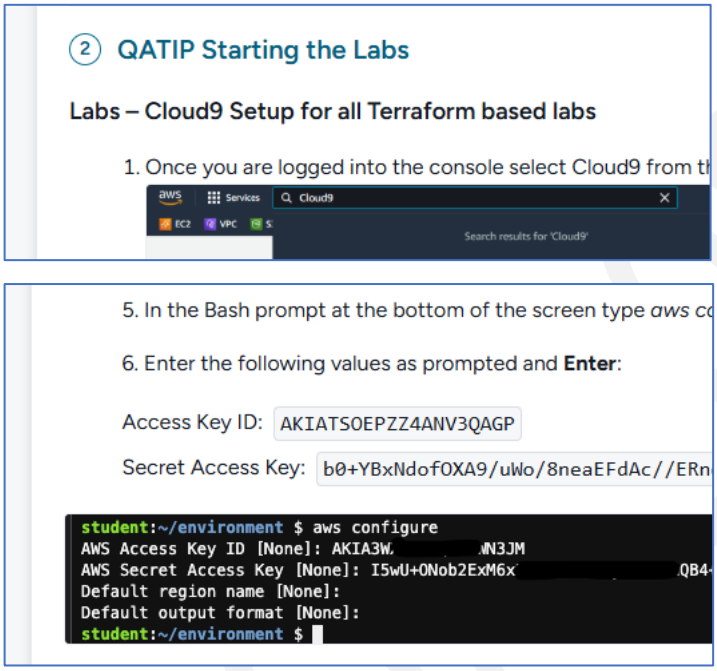
Rationale

- Preconditions catch bad input early; postconditions ensure that no unexpected transformations occur during deployment logic. This phase simulates bugs or misconfigurations that can pass initial validation but result in unintended infrastructure. Postconditions help catch these issues.

Steps

1. Update **terraform.tfvars**, changing the **bucket_name** variable value from "lab-bucket-xxxxxx" to "my-bucket-xxxxxx"
2. Run **terraform plan**
3. Terraform planning fails with a validation error (precondition at lines 36-39). The precondition checks that the variable **var.bucket_name** contains "lab-bucket"
4. Undo the changes to terraform.tfvars, reverting the bucket_name variable back to "lab-bucket-xxxx"
5. Run **terraform plan**
6. The planning should be successful as the preconditions are now met
7. You will now introduce a logic error that will be captured by a postcondition check
8. Modify line **28** of main.tf, replacing the 2nd instance of "**lab-bucket**" with "**test-bucket**". This is to simulate a logic error whereby the bucket name is changed from the value supplied as a variable (which passes the precondition check) resulting in the actual bucket name being something other than intended. The postcondition check validates the actual name that would be given to the bucket once all terraform logic is applied.
9. Run **terraform plan**
10. Terraform fails with a validation error (postcondition at lines 41-45).
11. Revert the changes to line **28**, re-aligning the actual bucket name back to the name derived from the variable.
12. Run **terraform plan**

13. Verify successful deployment with **terraform apply**
14. Switch to UI to verify EC2 instance and S3 bucket are created
15. Clean up with **terraform destroy**
16. Revert to using your **student** credentials in Cloud9 by running **aws configure** again and supplying your 'student' access key and secret, as shown in the "Starting the Labs" instructions:



Lab Summary Reflection: Validations and Role-Based Access

In this lab, you explored key DevOps principles around **security, access control, and configuration validation** in a real-world AWS Terraform deployment context.

What You Demonstrated

Secure Identity Management:
You created a least-privilege IAM user and configured Terraform to assume a tightly scoped role. This mirrors best practices in enterprise environments where users never have direct access to sensitive resources.

AssumeRole in Action:
You experienced firsthand how Terraform fails without role assumption — reinforcing the principle of least privilege and the need for controlled privilege elevation.

Terraform Preconditions and Postconditions:
You implemented both:

- **Preconditions** to confirm user input and enforce naming/instance standards.
- **Postconditions** to catch unexpected results or logic errors that could lead to misconfigured infrastructure.

Why This Matters

In production environments:

- Misapplied permissions can expose resources or break automation.
- Terraform plans may appear valid but still deploy unintended resources due to logic flaws.
- Role-based access enables secure collaboration across teams, while validation provides safety nets against human error.
- This lab hopefully helped build your confidence in:
    - Structuring secure Terraform deployments.
    - Writing and understanding Terraform validation blocks.

***** Congratulations, you have completed this lab *****